

Financial Portfolio Management System

A Database Management Systems Project

Student

Manasa Konala |
AP24110010943

Course: Database Management Systems





Introduction

Why Portfolio Management Matters

Investors manage diverse assets — stocks, bonds, mutual funds, and real estate — requiring structured, reliable oversight to maximize returns and minimize risk.

The Problem with Manual Tracking

Spreadsheet-based tracking is error-prone, lacks real-time updates, and cannot scale to handle complex multi-asset portfolios or historical trend analysis.

The Case for a DBMS Solution

A relational database enforces data integrity, enables complex queries, and supports scalable reporting — transforming raw financial data into actionable intelligence.

Project Objectives



Portfolio Tracking

Monitor asset allocations and portfolio value in real time across multiple investment categories.



Asset Management

Maintain structured records of stocks, bonds, mutual funds, and real estate with full CRUD support.



Risk Analysis

Profile investor risk tolerance and map it against asset volatility for informed decision-making.



Reporting via SQL Views

Generate portfolio summaries and risk reports using pre-defined SQL views for consistent output.



Historical Price Analysis

Store and query time-series asset price data to identify trends and support investment forecasting.

Technology Stack



SQL

DDL, DML, DCL — full relational query language for schema definition and data manipulation.



MySQL

Open-source RDBMS used as the primary database engine for all schema and query execution.



ER Modeling

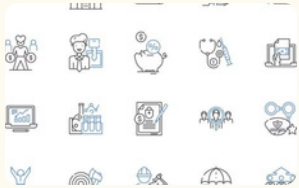
Entity-Relationship diagrams capture data structure and inter-table relationships visually.



Python

Used for database connectivity, data processing pipelines, and automated report generation via libraries like mysql - connector.

Key Features



Multi-Asset Tracking

Unified tracking across stocks, bonds, mutual funds, and real estate within a single schema.



Transaction Management

Full transaction history with buy/sell records, timestamps, and broker association via foreign keys.



Risk Profiling

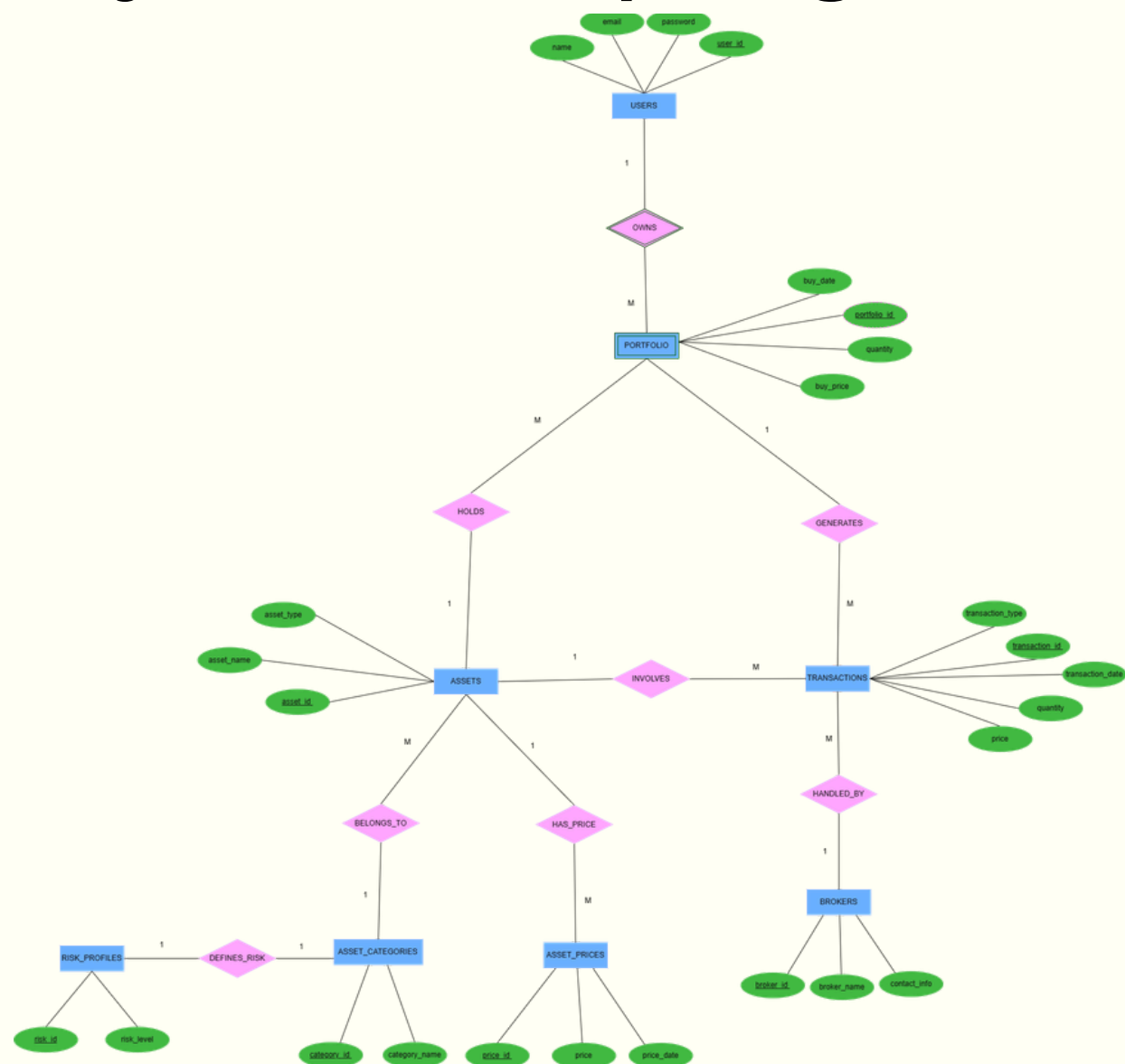
Each user is assigned a risk profile, enabling tailored asset recommendations and exposure limits.



Price History

ASSET_PRICES table captures date-stamped closing prices for longitudinal performance analysis.

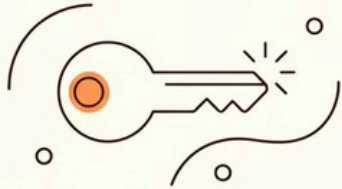
Entity-Relationship Diagram



Entities & Relationships
Represents all core entities: USERS, PORTFOLIO, ASSETS, TRANSACTIONS, BROKERS, ASSET_PRICES, RISK_PROFILES, and ASSET_CATEGORIES — and their associations.

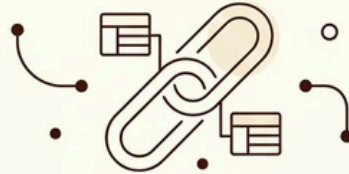
Integrity Constraints
Primary keys uniquely identify each record. Foreign keys enforce referential integrity across all related tables, preventing orphaned or inconsistent data entries.

Database Design



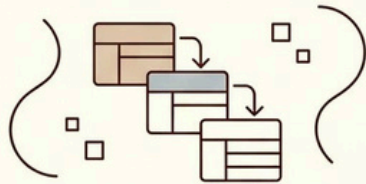
PRIMARY KEYS

Uniquely identify each record in USERS, ASSETS, PORTFOLIO, and TRANSACTIONS tables.



FOREIGN KEYS

Link PORTFOLIO to USERS table, and TRANSACTIONS to ASSETS and BROKERS.



REFERENTIAL INTEGRITY

Ensures database consistency; cascading constraints prevent creating orphaned records.

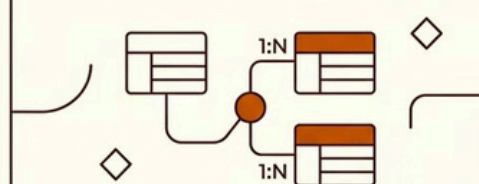


TABLE RELATIONSHIPS

Define connections, like one-to-many between USERS and PORTFOLIO, or ASSETS and TRANSACTIONS.

Schema Highlights

- **USERS** → linked to PORTFOLIO and RISK_PROFILES
- **ASSETS** → categorized via ASSET_CATEGORIES; priced via ASSET_PRICES
- **TRANSACTIONS** → references PORTFOLIO, ASSETS, and BROKERS
- **BROKERS** → associated with each executed transaction

📌 All foreign key constraints are enforced at the schema level using MySQL InnoDB engine for full referential integrity support.

Normalization

1

1NF — First Normal Form

All attributes contain atomic values. No repeating groups. Each column holds a single data type.

2

2NF — Second Normal Form

Achieves 1NF and eliminates partial dependencies. All non-key attributes fully depend on the primary key.

3

3NF — Third Normal Form

Achieves 2NF and removes transitive dependencies — no non-key attribute depends on another non-key attribute.

Reduced Redundancy

Asset categories and risk profiles are stored once and referenced via foreign keys — eliminating duplicated data across rows.

Improved Consistency

Normalization ensures that updating a broker name or asset category propagates correctly without anomalies.

SQL Query Examples

1

JOIN Query

```
SELECT u.name, a.asset_name,  
       t.quantity  
FROM USERS u  
JOIN PORTFOLIO p ON  
  u.user_id = p.user_id  
JOIN TRANSACTIONS t ON  
  p.portfolio_id = t.portfolio_id  
JOIN ASSETS a ON t.asset_id =  
  a.asset_id;
```

2

Aggregate Query

```
SELECT a.asset_name,  
       AVG(ap.closing_price) AS  
       avg_price  
FROM ASSETS a  
JOIN ASSET_PRICES ap ON  
  a.asset_id = ap.asset_id  
GROUP BY a.asset_name;
```

3

Subquery

```
SELECT name FROM USERS  
WHERE user_id IN (  
  SELECT user_id FROM  
  PORTFOLIO  
  WHERE portfolio_id IN (  
    SELECT portfolio_id FROM  
    TRANSACTIONS  
    WHERE quantity > 100));
```

SQL Views & Conclusion

Views Implemented

portfolio_summary_view — Aggregates each user's total holdings, asset types, and current portfolio valuation in a single queryable view.

asset_risk_view — Joins ASSETS with RISK_PROFILES to surface each asset's risk classification alongside its current price and category.

Project Outcomes

- Structured, scalable management of multi-asset investment portfolios
- Normalized 3NF schema ensuring data consistency and integrity
- Reusable SQL views enabling instant, dashboard-ready reporting
- Robust transaction and price-history tracking for analytical queries

✓ This system demonstrates how a well-designed DBMS transforms raw financial data into structured, queryable intelligence — ready for real-world deployment.